# Bài 1.8 – Toán: vector và ma trận

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *So sánh ví von*

## 🎯 Sau bài này bạn sẽ

- Hiểu vì sao trong ML, **mọi thứ đều là vector**: căn nhà, khách hàng, bức ảnh, câu văn.
- Tính được **tích vô hướng** và thấy nó chính là "công thức dự đoán" của nhiều mô hình.
- Đo **độ giống nhau** giữa hai vector (khoảng cách, cosine), là nền tảng của hệ gợi ý.
- Nhân ma trận để dự đoán **cả nghìn mẫu trong một phép tính**.
- Đọc và sửa lỗi **shape không khớp**.

**Chia buổi:**

- **Buổi 1:** Mục 1 → 4 (vector, phép toán, tích vô hướng).
- **Buổi 2:** Mục 5 → 9 (độ giống nhau, ma trận, nhân ma trận, bài tập).

> Không cần nhớ định lý hay chứng minh. Ta chỉ học **những phép toán ML thật sự dùng**, mỗi phép đi kèm một việc cụ thể nó làm.

---

## 1. So sánh ví von: Vector là "hồ sơ" bằng số

Hãy tưởng tượng bạn là nhân viên môi giới bất động sản. Mỗi căn hộ có một **tờ hồ sơ**:

```
┌──────────────────────────┐
│ HỒ SƠ CĂN HỘ A           │
│ Diện tích:     60 m²     │
│ Phòng ngủ:      2        │
│ Cách trung tâm: 5 km     │
└──────────────────────────┘
```

Máy tính không đọc được "tờ hồ sơ". Nó cần một **dãy số theo thứ tự cố định**:

```
căn A = [60, 2, 5]
         │   │  └─ km tới trung tâm
         │   └──── số phòng ngủ
         └──────── diện tích
```

Đó là một **vector**. Mọi thứ trong ML đều được biến thành "hồ sơ bằng số" như vậy:

| Đối tượng | Vector |
|---|---|
| Căn nhà | [diện tích, số phòng, khoảng cách, ...] |
| Khách vay tiền | [tuổi, thu nhập, số khoản nợ, ...] |

| Đối tượng | Vector |
|---|---|
| Khẩu vị một người | [độ thích cay, ngọt, mặn, béo] |
| Ảnh đen trắng 28×28 | 784 số, mỗi số là độ sáng một điểm ảnh (bài 3.13) |
| Một từ tiếng Việt | 300 số mô tả "nghĩa" của từ (bài NLP2.1) |

Khi đã là vector, ta có thể **cộng, so sánh, đo khoảng cách** giữa các căn nhà, các khách hàng, các từ ngữ, bằng cùng một bộ công cụ toán.

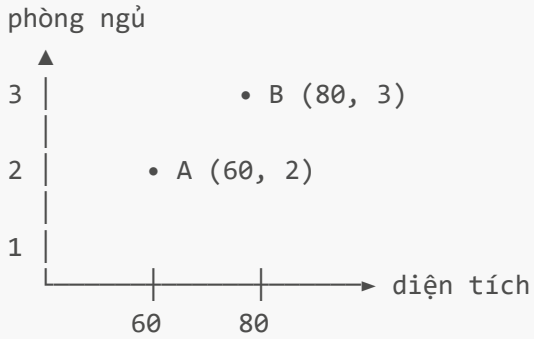
---

# 2. Vector

## 2.1. Khái niệm

- **Vector:** Một dãy số có thứ tự. Số lượng phần tử gọi là **số chiều** (dimension).
- **Ví dụ đời thực:** Tọa độ GPS [vĩ độ, kinh độ] là vector 2 chiều. Căn A là vector 3 chiều.
- **Feature vector (Vector đặc trưng):** vector mô tả một mẫu dữ liệu. Chính là **một dòng** trong bảng dữ liệu.

## 2.2. Hình dung vector 2 chiều




Mỗi vector là một **điểm** (hoặc một mũi tên từ gốc tọa độ tới điểm đó). Với 3 chiều ta còn hình dung được. Với 784 chiều thì không, nhưng **toán vẫn y hệt**.

---

# 3. Phép toán cơ bản trên vector

```
import numpy as np

can_a = np.array([60, 2, 5])   # [diện tích, phòng ngủ, km tới trung tâm]
can_b = np.array([80, 3, 2])

print(can_a + can_b,   # cộng từng cặp phần tử
      can_b - can_a,   # trừ từng cặp: B hơn A bao nhiêu ở mỗi tiêu chí
      2 * can_a)       # nhân vô hướng: nhân mọi phần tử với 2
```

**Kết quả mẫu:**

```
[140   5   7] [20  1 -3] [120   4  10]
```

**Ý nghĩa của `can_b - can_a = [20, 1, -3]`:** căn B rộng hơn 20 $m^2$, nhiều hơn 1 phòng, gần trung tâm hơn 3 km. Hiệu hai vector cho biết **chúng khác nhau thế nào**, dùng ở mục 5.

Bạn đã làm những phép này ở bài 1.3. NumPy chính là "máy tính vector".

---

# 4. Tích vô hướng (Dot product): trái tim của ML

## 4.1. Vấn đề

Bạn muốn ước lượng giá căn hộ từ hồ sơ. Kinh nghiệm cho bạn "công thức" kiểu:

- Mỗi $m^2$ thêm **0,05 tỷ**.
- Mỗi phòng ngủ thêm **0,3 tỷ**.
- Mỗi km xa trung tâm **trừ 0,1 tỷ**.
- Cộng thêm một khoản cơ bản **0,5 tỷ**.

## 4.2. Khái niệm

- **Dot product (Tích vô hướng):** Nhân từng cặp phần tử tương ứng của hai vector, rồi **cộng tất cả lại** → ra **một con số**.
- **Ví dụ đời thực:** Tính tổng tiền đi chợ: [số lượng từng món] · [đơn giá từng món] = tổng tiền.

**Công thức:**

$$w \cdot x = w_1x_1 + w_2x_2 + ... + w_nx_n$$

| Ký hiệu | Ý nghĩa |
|---|---|
| x | Vector đặc trưng (hồ sơ căn nhà) |
| w | Vector **trọng số** (weight): mức độ quan trọng của từng đặc trưng |
| w · x | Tích vô hướng, ra một số |

## 4.3. Tính thử

```
w = [ 0.05,  0.3,  -0.1 ]     (trọng số)
x = [ 60  ,  2  ,   5   ]     (căn A)
      ↓      ↓      ↓
     3.0  + 0.6  + (-0.5)  = 3.1
```

Cộng khoản cơ bản b = 0,5: giá dự đoán = 3,1 + 0,5 = **3,6 tỷ**.

```
w = np.array([0.05, 0.3, -0.1])   # trọng số
b = 0.5                           # khoản cơ bản (gọi là bias, hệ số chặn)
x = np.array([60, 2, 5])          # căn A

print(w * x,                # nhân từng cặp
      (w * x).sum(),        # rồi cộng lại = tích vô hướng
      np.dot(w, x),         # cách viết bằng hàm
      w @ x)                # cách viết bằng toán tử @ (gọn nhất, khuyên dùng)
print("Giá dự đoán:", w @ x + b)
```

**Kết quả mẫu:**

```
[ 3.   0.6 -0.5] 3.1 3.1 3.1
Giá dự đoán: 3.6
```

### 4.4. Vì sao gọi là "trái tim của ML"?

Công thức $\hat{y} = w \cdot x + b$ chính là:

- **Linear Regression (Hồi quy tuyến tính)** (bài 2.2): dự đoán giá nhà.
- Phần lõi của **Logistic Regression** (bài 2.4): dự đoán spam hay không.
- **Một nơ-ron** trong mạng nơ-ron (bài 3.1).

Điểm khác biệt quan trọng: ở trên, **bạn tự nghĩ ra** w = [0,05; 0,3; −0,1]. Trong ML, **máy tự tìm** w tốt nhất từ dữ liệu. Cách máy tìm w là chủ đề của bài 1.9 (Gradient Descent).

> Ký hiệu **ŷ** (đọc "y mũ") nghĩa là **giá trị dự đoán**, phân biệt với **y** là giá trị thật.

---

## 5. Đo độ giống nhau giữa hai vector

### 5.1. Vấn đề: gợi ý món ăn

App giao đồ ăn chấm điểm khẩu vị mỗi người dùng theo 4 tiêu chí (thang 0–5): **[cay, ngọt, mặn, béo]**.

```
an   = np.array([5, 1, 3, 2])   # An: rất thích cay
binh = np.array([4, 2, 3, 1])   # Bình: cũng thích cay
chi  = np.array([1, 5, 1, 4])   # Chi: thích ngọt, béo
```

Muốn gợi ý cho An các món mà người **có khẩu vị giống An** hay gọi. Ai giống An hơn: Bình hay Chi?

### 5.2. Khoảng cách Euclid (Euclidean distance)

- **Định nghĩa:** Độ dài đoạn thẳng nối hai điểm. Chính là "khoảng cách thước kẻ".

- **Ví dụ đời thực:** Khoảng cách chim bay giữa hai địa điểm trên bản đồ.

**Công thức:**

$$d(u, v) = \sqrt{[\ (u_1 - v_1)^2 + (u_2 - v_2)^2 + \ldots + (u_n - v_n)^2\ ]}$$

Đây là định lý Pytago, mở rộng cho nhiều chiều.

**Tính thử An – Bình:**

- Hiệu: [5−4, 1−2, 3−3, 2−1] = [1, −1, 0, 1]
- Bình phương: [1, 1, 0, 1] → tổng = 3
- d = √3 ≈ **1,732**

**Tính thử An – Chi:**

- Hiệu: [4, −4, 2, −2] → bình phương: [16, 16, 4, 4] → tổng = 40
- d = √40 ≈ **6,325**

```
# np.linalg.norm(v): độ dài của vector v. Độ dài của (u - v) = khoảng cách giữa u và v
print(np.linalg.norm(an - binh).round(3), np.linalg.norm(an - chi).round(3))
```

**Kết quả mẫu:**

```
1.732 6.325
```

Bình gần An hơn nhiều → gợi ý cho An các món Bình thích. Ý tưởng "tìm những điểm gần nhất" chính là thuật toán **KNN – K-Nearest Neighbors (K láng giềng gần nhất)** (sẽ học kỹ ở bài 2.7).

## 5.3. Cosine similarity (Độ tương đồng cosine)

**Vấn đề của khoảng cách Euclid:** Dũng có khẩu vị **y hệt** An, nhưng Dũng hay chấm điểm "rộng tay" theo thang 10:

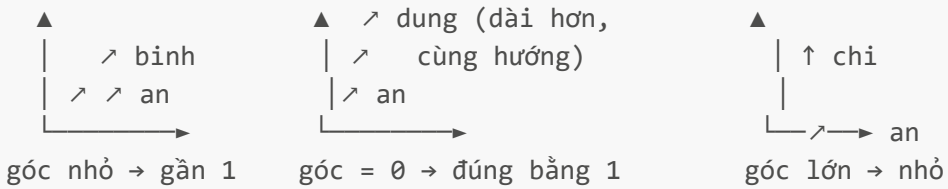
```
dung = np.array([10, 2, 6, 4])      # đúng bằng 2 × An
print(np.linalg.norm(an - dung).round(3))
```

**Kết quả mẫu:**

```
6.245
```

Khoảng cách 6,245, xa gần bằng Chi! Nhưng rõ ràng Dũng và An cùng "gu". Ta cần một thước đo chỉ quan tâm đến **hướng** (tỉ lệ giữa các tiêu chí), không quan tâm **độ lớn**.

- **Cosine similarity:** Đo **góc** giữa hai vector. Cùng hướng → 1. Vuông góc (không liên quan) → 0. Ngược hướng → −1.




**Công thức:**

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \times \|v\|}$$

| Ký hiệu | Ý nghĩa |
|---|---|
| $u \cdot v$ | Tích vô hướng (mục 4) |
| $\|u\|$ | Độ dài (norm) của u = $\sqrt{(u_1^2 + u_2^2 + \ldots)}$ |

**Tính thử An – Bình:**

- An · Bình = 5×4 + 1×2 + 3×3 + 2×1 = 20 + 2 + 9 + 2 = **33**
- ||An|| = √(25 + 1 + 9 + 4) = √39 ≈ 6,245
- ||Bình|| = √(16 + 4 + 9 + 1) = √30 ≈ 5,477
- cos = 33 / (6,245 × 5,477) = 33 / 34,205 ≈ **0,965**

```
def cosine(u, v):
    return u @ v / (np.linalg.norm(u) * np.linalg.norm(v))

print(round(cosine(an, binh), 3), round(cosine(an, chi), 3), round(cosine(an,
dung), 3))
```

**Kết quả mẫu:**

```
0.965 0.513 1.0
```

Cosine An – Dũng = **1,0** tuyệt đối, đúng là cùng gu. Cosine similarity là thước đo **cực kỳ phổ biến** trong:

- Hệ gợi ý (sản phẩm, phim, món ăn).

- Tìm văn bản giống nhau, tìm kiếm ngữ nghĩa, chatbot RAG (sẽ học kỹ ở bài NLP4.2).

### 5.4. Chọn thước đo nào?

| | **Euclid** | **Cosine** |
|---|---|---|
| Đo | Khoảng cách (càng nhỏ càng giống) | Góc (càng gần 1 càng giống) |
| Quan tâm độ lớn? | Có | Không, chỉ quan tâm hướng |
| Dùng khi | Độ lớn có ý nghĩa (diện tích, tuổi) | Tỉ lệ/xu hướng quan trọng hơn (văn bản, khẩu vị) |

## 6. Ma trận: cả bảng dữ liệu

### 6.1. Khái niệm

- **Matrix (Ma trận):** Một bảng số có hàng và cột. Có thể xem là nhiều vector xếp chồng lên nhau.
- **Ví dụ đời thực:** Bảng Excel chỉ chứa số.
- Trong ML, **ma trận X** = toàn bộ dữ liệu: mỗi **hàng** là một mẫu (một feature vector), mỗi **cột** là một đặc trưng. Kích thước ghi là **(số hàng × số cột)**.

```
X = np.array([
    [60,  2, 5],    # căn A
    [80,  3, 2],    # căn B
    [45,  1, 8],    # căn C
    [100, 3, 1],    # căn D
])
print(X.shape)       # 4 căn × 3 đặc trưng
```

**Kết quả mẫu:**

```
(4, 3)
```

### 6.2. Ma trận × vector: dự đoán cả bảng một lúc

Thay vì tính `w @ x + b` cho từng căn bằng vòng lặp:

```
print(X @ w + b)    # dự đoán giá cho CẢ 4 căn trong một phép tính
```

**Kết quả mẫu:**

```
[3.6  5.2  2.25 6.3 ]
```

**Chuyện gì xảy ra bên trong?** Mỗi **hàng** của X được nhân vô hướng với w:

```
        X (4×3)              w (3,)       X @ w (4,)     + b
┌                 ┐       ┌      ┐      ┌      ┐
│  60    2    5   │       │ 0.05 │      │ 3.1  │       3.6   ← căn A
│  80    3    2   │   @   │ 0.3  │   =  │ 4.7  │  +0.5 5.2   ← căn B
│  45    1    8   │       │ -0.1 │      │ 1.75 │       2.25  ← căn C
│ 100    3    1   │       └      ┘      │ 5.8  │       6.3   ← căn D
└                 ┘                    └      ┘

  Hàng 2: 80×0.05 + 3×0.3 + 2×(−0.1) = 4 + 0.9 − 0.2 = 4.7
```

Với 1 triệu căn nhà, đây vẫn chỉ là **một dòng code**, và chạy cực nhanh (bài 1.3). Đây là lý do ma trận có mặt khắp nơi trong ML.

### 6.3. Chuyển vị (Transpose)

- **Định nghĩa:** Lật ma trận: hàng thành cột, cột thành hàng. Ký hiệu $X^T$, trong NumPy là `X.T`.

```
print(X.T)
print(X.T.shape)
```

**Kết quả mẫu:**

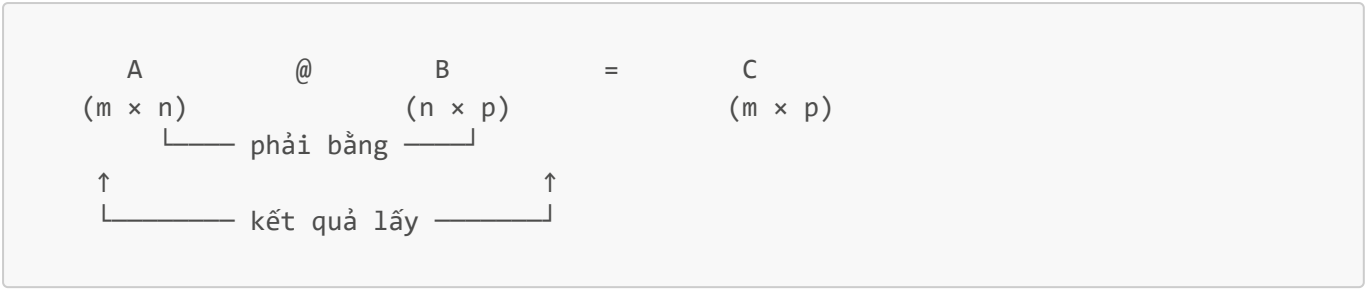
```
[[ 60  80  45 100]
 [  2   3   1   3]
 [  5   2   8   1]]
(3, 4)
```

Bạn sẽ gặp `.T` thường xuyên khi cần "xoay" dữ liệu cho khớp kích thước.

---

## 7. Nhân hai ma trận và quy tắc shape

### 7.1. Quy tắc vàng




**Số cột của A phải bằng số hàng của B.** Kết quả có số hàng của A và số cột của B.

**Cách tính:** phần tử ở hàng i, cột j của C = **tích vô hướng** của hàng i của A với cột j của B.

## 7.2. Tính thử

```
A (3×2)        B (2×3)
┌      ┐      ┌          ┐
│ 1  2 │      │  7  8  9 │
│ 3  4 │  @   │ 10 11 12 │
│ 5  6 │      └          ┘
└      ┘

C[0,0] = hàng 0 của A · cột 0 của B = 1×7 + 2×10 = 27
C[1,2] = hàng 1 của A · cột 2 của B = 3×9 + 4×12 = 27 + 48 = 75
```

```python
A = np.array([[1, 2], [3, 4], [5, 6]])          # 3×2
B = np.array([[7, 8, 9], [10, 11, 12]])         # 2×3
print(A @ B)
print((A @ B).shape)
```

**Kết quả mẫu:**

```
[[ 27  30  33]
 [ 61  68  75]
 [ 95 106 117]]
(3, 3)
```

## 7.3. Lỗi kinh điển: shape không khớp

```python
try:
    A @ A          # (3×2) @ (3×2): số cột A (2) ≠ số hàng A (3)
except ValueError as e:
    print("Lỗi:", e)
```

**Kết quả mẫu:**

```
Lỗi: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc
signature (n?,k),(k,m?)->(n?,m?) (size 3 is different from 2)
```

Đọc lỗi: "size 3 is different from 2" = kích thước 3 và 2 không khớp. **Cách sửa đầu tiên luôn là in `.shape` của cả hai**, rồi xem có cần `.T` hay `.reshape` không. Bạn sẽ gặp lỗi kiểu này rất nhiều ở PyTorch (bài 3.8).

### 7.4. `@` khác `*`

```
print(A * A)     # * : nhân TỪNG PHẦN TỬ tương ứng (cần cùng shape)
```

**Kết quả mẫu:**

```
[[ 1  4]
 [ 9 16]
 [25 36]]
```

| Toán tử | Làm gì | Điều kiện shape |
|---|---|---|
| `*` | Nhân từng phần tử | Cùng shape (hoặc broadcast được, bài 1.3) |
| `@` | Nhân ma trận (tích vô hướng hàng × cột) | Số cột trái = số hàng phải |

**Mạng nơ-ron là gì?** Về mặt tính toán, nó là **chuỗi các phép nhân ma trận** xen kẽ với vài hàm đơn giản (sẽ học kỹ ở bài 3.2–3.3). GPU (bài 0.2) mạnh chính vì nó nhân ma trận cực nhanh.

---

## 8. Bài tập

1. Với w = [0,06; 0,2; −0,15], b = 0,3, dự đoán giá 3 căn sau trong **một** phép tính: `nha = np.array([[50, 2, 3], [70, 3, 6], [90, 3, 2]])`. Kiểm tra căn đầu bằng tay.
2. Cho u = [1, 2, 3], v = [4, −5, 6]. Tính **bằng tay** rồi kiểm tra bằng NumPy: u · v, ||u||, cos(u, v).
3. Cho M (3×2) và N (2×3) dưới đây. Đoán shape của `M @ N` và `N @ M` **trước khi chạy**. Hai kết quả có bằng nhau không?

   ```
   M = np.array([[1, 0], [0, 2], [3, 1]])
   N = np.array([[2, 1, 0], [1, 3, 1]])
   ```

4. **Mini hệ gợi ý.** Mỗi món ăn có vector [cay, ngọt, mặn, béo]:

   ```
   mon = {"Phở bò": [2, 1, 4, 3], "Bún bò Huế": [5, 1, 4, 3],
          "Chè": [0, 5, 0, 2], "Gỏi cuốn": [1, 2, 2, 1]}
   ```

Tính cosine giữa khẩu vị của An `[5, 1, 3, 2]` với từng món. Nên gợi ý món nào đầu tiên?

### Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
# 1
nha = np.array([[50, 2, 3], [70, 3, 6], [90, 3, 2]])
w2 = np.array([0.06, 0.2, -0.15]); b2 = 0.3
print(nha @ w2 + b2)          # [3.25 4.2  6.  ]
# Tay: 50×0.06 + 2×0.2 + 3×(−0.15) + 0.3 = 3 + 0.4 − 0.45 + 0.3 = 3.25 ✓

# 2
u = np.array([1, 2, 3]); v = np.array([4, -5, 6])
print(u @ v, np.linalg.norm(u).round(3), round(cosine(u, v), 3))   # 12 3.742
0.365
# Tay: u·v = 4 − 10 + 18 = 12. ‖u‖ = √14 ≈ 3.742. ‖v‖ = √77 ≈ 8.775.
#      cos = 12 / (3.742 × 8.775) ≈ 12 / 32.83 ≈ 0.365

# 3
print((M @ N).shape, (N @ M).shape)     # (3, 3) (2, 2)
# Khác shape nên chắc chắn khác nhau. Nhân ma trận KHÔNG có tính giao hoán: A@B ≠
B@A.

# 4
for ten, vec in mon.items():
    print(ten, round(cosine(an, np.array(vec)), 3))
# Phở bò 0.848 | Bún bò Huế 0.987 | Chè 0.268 | Gỏi cuốn 0.76
# → Gợi ý Bún bò Huế (cay, đậm đà) đầu tiên. Chè xếp cuối.
```

---

## 9. Flashcard

| Mặt trước | Mặt sau |
|---|---|
| Feature vector là gì? | Dãy số mô tả một mẫu dữ liệu = một dòng trong bảng. |
| Tích vô hướng tính thế nào? | Nhân từng cặp phần tử rồi cộng lại, ra 1 số. |
| Công thức dự đoán tuyến tính? | ŷ = w · x + b |
| w và b gọi là gì? | w: trọng số. b: bias (hệ số chặn). |
| Khoảng cách Euclid? | √(tổng bình phương hiệu từng phần tử). |
| Khi nào dùng cosine thay vì Euclid? | Khi hướng/tỉ lệ quan trọng hơn độ lớn (văn bản, khẩu vị). |
| `X @ w` với X (1000×3), w (3,) ra gì? | 1000 dự đoán, mỗi hàng một số. |
| Điều kiện nhân A @ B? | Số cột A = số hàng B. Kết quả: (hàng A × cột B). |
| `*` vs `@`? | `*` nhân từng phần tử. `@` nhân ma trận. |

| Mặt trước | Mặt sau |
|---|---|
| Gặp lỗi shape, làm gì đầu tiên? | In `.shape` của mọi mảng liên quan. |

**Bài tiếp theo:** 1.9 – Toán: đạo hàm, gradient, Gradient Descent.